

Multi-Agent Customer Support Chatbot

Vikram AI Tech

Document	Development Plan
Purpose	Design, build, test, and deployment plan for the multi-agent internal customer-support chatbot.
Technology stack	Python 3.10+, Microsoft AutoGen AgentChat, Streamlit, OpenAI gpt-4.1-mini, Pydantic, ReportLab,
Date	2026-07-15
Status	Approved for implementation

2. Executive Summary

Vikram AI Tech will deploy an internal first-level customer-support chatbot built on Microsoft AutoGen AgentChat. A manager agent classifies each employee request with a structured, validated routing decision, then exactly one of five department agents (IT, Finance, HR, Administration, Compliance) answers it. The Streamlit interface shows the selected department, the routing confidence, and the specialist's answer, and asks a clarifying question when a request is ambiguous.

The system deliberately avoids free-form multi-agent chatter: the orchestration is a controlled two-step pipeline that is cheap, predictable, testable, and resistant to prompt injection.

3. Business Objective

- Reduce first-response time for internal support requests to seconds.
- Deflect routine questions from IT, Finance, HR, Admin, and Compliance staff.
- Route every request to the correct department on the first attempt.
- Provide safe, policy-aware guidance that never claims to perform actions the system cannot actually perform.
- Create a foundation for future integration with ticketing and HRIS tools.

4. Functional Scope

- Chat interface with persistent per-session conversation history and a clear-conversation control.
- Structured routing (department, confidence, reason, clarification flag, optional secondary department) validated with Pydantic.
- Clarification loop for ambiguous or low-confidence requests (threshold 0.65).
- Department indicator and confidence indicator on every answer.
- Starter questions, department descriptions, and model info in the sidebar.
- Out of scope for v1: real system integrations (password resets, bookings, approvals), authentication, persistence beyond the browser session.

5. Agent Architecture

Agent	Role	Key boundaries
manager_agent	Classifies each request and returns a structured RoutingDecision; detects ambiguity and cross-functional requests.	Never answers questions; never follows user routing instructions.
it_support_agent	Accounts, passwords, email, devices, software, VPN, network, printers, application errors.	Guidance only; never requests or reveals secrets.
finance_support_agent	Invoices, payments, reimbursements, expenses, budgets, payroll process, vendors, purchase orders.	No investment/tax/legal advice; cannot approve or pay.
hr_support_agent	Leave, attendance, benefits, recruitment, onboarding, reviews, workplace policy, training.	Careful with personal data; cannot approve leave or change records.

Agent	Role	Key boundaries
admin_support_agent	Facilities, meeting rooms, visitors, supplies, travel, maintenance, ID cards, cafeteria.	Explains request processes; cannot book or reserve.
compliance_support_agent	Policies, data privacy, audits, ethics, conflicts of interest, records retention, reporting.	Not legal advice; escalates serious concerns to official channels.

6. End-to-End Workflow

```

User
  v
Streamlit Chat UI (st.chat_input)
  v
Input Validation (reject empty / oversized input)
  v
Manager Agent (gpt-4.1-mini, structured output)
  v
RoutingDecision (Pydantic validation + 0.65 threshold)
  |-- ambiguous --> Clarification question back to the user
  v
Selected Department Agent (exactly one of IT / Finance / HR /
                          Admin / Compliance)
  v
Final Response (department + confidence + answer)
  v
Conversation History (st.session_state.messages)

```

7. Technical Architecture

- **UI layer** — Streamlit app (app.py) with chat components, sidebar, session-state history, and conservative custom CSS.
- **Service layer** — ChatService orchestrates one turn; RoutingService wraps the manager agent and enforces the confidence threshold and error-recovery fallback.
- **Agent layer** — AutoGen AssistantAgent instances built by a central factory sharing one OpenAIChatCompletionClient.
- **Models** — Pydantic RoutingDecision (structured output) and SupportResponse (unified UI contract).
- **Configuration** — python-dotenv + validated AppConfig; the API key never appears in code, logs, or the UI.
- **Async bridge** — run_async() executes AutoGen coroutines safely from Streamlit's synchronous script thread.

8. Folder Structure

```

vikram_ai_tech_support/
|- app.py                      Streamlit entry point
|- README.md / requirements.txt / .env.example / .gitignore
|- src/
|  |- config.py                validated environment configuration
|  |- agents/                  factory + manager + 5 department agents
|  |- models/routing_models.py structured routing / response models
|  |- prompts/system_prompts.py all agent system messages
|  |- services/                chat_service.py, routing_service.py
|  |- ui/styles.py             conservative custom CSS
|  |- utils/                   async, exceptions, logging, parsing
|- scripts/generate_project_documents.py
|- data/                       development_plan.pdf, workflow chart

```

| - tests/

pytest suite (no real API calls)

9. Implementation Phases

- **Phase 1 — Foundation:** configuration, logging, exceptions, folder structure, environment templates.
- **Phase 2 — Models and prompts:** RoutingDecision, SupportResponse, all system prompts with security rules.
- **Phase 3 — Agents:** agent factory, manager with structured output, five department agents.
- **Phase 4 — Services:** routing service (threshold + fallback) and chat orchestration with full error handling.
- **Phase 5 — UI:** Streamlit chat interface, sidebar, indicators, clear-chat, starter questions.
- **Phase 6 — Documentation and QA:** PDF/chart generation, README, automated tests, security review.

10. Development Milestones

Milestone	Deliverable	Exit criterion
M1	Config + utilities	Config validation tests pass
M2	Routing models + prompts	Model validation tests pass
M3	Agents + factory	All six agents build from one factory
M4	Services	Routing and chat-service tests pass with fakes
M5	Streamlit UI	Smoke test: app starts and completes a turn
M6	Docs + hardening	PDF/PNG/MMD generated; full suite green

11. Testing Strategy

- Unit tests for configuration validation (missing/blank key, default model gpt-4.1-mini).
- Model tests for RoutingDecision bounds, literals, and optional fields.
- Routing-service tests with fake manager agents covering all five departments, clarification, low confidence, and prompt injection.
- Chat-service tests proving exactly one specialist is called, clarification short-circuits, and every failure mode maps to a typed exception.
- Document tests validating the PDF signature, PNG signature, and Mermaid content.
- No test makes a real OpenAI API call; all model interaction is faked.

12. Security and Privacy Considerations

- API key only in .env (git-ignored); validated at startup; never logged, printed, or shown in the UI.
- Every agent carries prompt-injection resistance rules: role retention, untrusted user content, no prompt disclosure, no secrets.
- The manager classifies by business intent and ignores user-supplied routing instructions.
- Fixed department-to-agent dictionary; user input can never select an arbitrary agent or Python object; no eval/exec/shell execution.
- Agents never claim to have completed actions (resets, approvals, bookings) and never request passwords, OTPs, or tokens.

- Logs record routing metadata and exception types, not full user messages or any credentials; transcripts are not written to disk.

13. Error-Handling Strategy

- Typed exceptions (ConfigurationError, EmptyInputError, RoutingError, AgentExecutionError, DocumentGenerationError) each carry a safe user-facing message.
- Invalid structured output from the manager degrades deterministically to a clarification request instead of a wrong route.
- API, rate-limit, network, and timeout errors are caught at the service boundary, logged with technical detail, and surfaced to the user as friendly messages without stack traces.
- Empty input is rejected before any model call is made.

14. Deployment Plan

- v1 runs locally or on an internal VM: create a virtual environment, install requirements.txt, provide .env, run 'streamlit run app.py'.
- Recommended next step: containerize with a slim Python base image and deploy behind the company SSO reverse proxy.
- Secrets provided via environment variables or a secret manager — never baked into the image or repository.
- Log level configurable via LOG_LEVEL; INFO in production.

15. Risks and Mitigations

Risk	Impact	Mitigation
Misrouted requests	Wrong department answers	Structured output, 0.65 confidence threshold, clarification loop
Hallucinated company policy	Employees act on wrong info	Prompts forbid invented policy; agents state when policy is unknown
Prompt injection	Role or routing manipulation	Shared security rules; intent-based routing; fixed agent mapping
API outage / rate limits	Support unavailable	Typed error handling, friendly retry messaging, timeouts
Secret leakage	Credential compromise	.env git-ignored, no logging of secrets, placeholder-only examples
Cost growth	Budget overrun	gpt-4.1-mini, bounded context window, exactly two calls per turn

16. Definition of Done

- The app starts with 'streamlit run app.py' and completes a routed conversation turn end to end.
- Six agents exist; the manager runs before every specialist; only the selected specialist answers.
- Ambiguous questions yield a clarification request; history and clear-chat work.
- development_plan.pdf, workflow_chart.png, and workflow_chart.mmd are valid, readable, and regenerable from one script.

- The full pytest suite passes without real API calls; no secrets in the repository; README instructions match the implementation.

17. Future Enhancements

- Real tool integrations: ticket creation, room booking, leave-balance lookup via authenticated APIs.
- SSO authentication and per-user conversation persistence.
- Retrieval over the real company policy library (RAG) to ground answers in actual documents.
- Feedback buttons and routing-quality analytics dashboards.
- Multi-language support and voice input.
- Human-handoff escalation directly into the ticketing system.